Class Activity:

Create a chart comparing procedural programming and OOP. Have students explain scenarios where OOP is more useful.

Procedural Oriented Programming	Object-Oriented Programming
In procedural programming, the program is divided into small parts called ***functions***.	In object-oriented programming, the program is divided into small parts called ***objects***.
There is no access specifier in procedural programming.	Object-oriented programming has access specifiers like private, public, protected, etc.
Adding new data and functions is not easy.	Adding new data and function is easy.
Procedural programming does not have any proper way of hiding data so it is ***less secure***.	Object-oriented programming provides data hiding so it is ***more secure***.
In procedural programming, there is no concept of data hiding and inheritance.	In object-oriented programming, the concept of data hiding and inheritance is used.
In procedural programming, the function is more important than the data.	In object-oriented programming, data is more important than function.
Procedural programming is based on the ***unreal world***.	Object-oriented programming is based on the ***real world***.
Examples: C, FORTRAN, Pascal, Basic, etc.	**Examples:** C++, Java, Python, C#, etc.

Scenarios Where OOP is More Useful

Students should discuss and explain the following situations:

1. School Management System

- **Entities**: Student, Teacher, Class, Exam, Attendance
- **Why OOP?** Each entity has its own data and behaviour. Inheritance can be used (e.g., Person → Student/Teacher). Encapsulation protects sensitive data like marks and fees.

2. Banking Application

- **Entities**: Account, Customer, Transaction, Loan
- **Why OOP?** Balance must be private (Encapsulation). Different account types (Savings, Current) can inherit from a base Account class. Polymorphism allows different transaction methods.

3. Online Shopping Website / E-commerce

- **Entities**: Product, Customer, Cart, Order, Payment
- **Why OOP?** Products can be of different types (Electronics, Clothing) using inheritance. Payment methods (Card, UPI, Wallet) use polymorphism with the same method name processPayment().

4. Library Management System

- **Entities**: Book, Member, Librarian, Issue/Return
- **Why OOP?** Easy to manage multiple books and members as objects. Inheritance can create categories of books. Code is reusable and easy to maintain.

5. Game Development (e.g., Car Racing Game)

- **Entities**: Car, Player, Obstacle, Track
- **Why OOP?** Different cars can inherit from a base Vehicle class. Polymorphism allows different vehicles to have the same method (accelerate()) but different behaviour.

6. Hospital Management System

- **Entities**: Patient, Doctor, Appointment, Bill
- **Why OOP?** Real-world entities map directly to classes. Sensitive patient data is protected through encapsulation. Easy to extend later (add new departments).